

Python-Based Freestyle Project in Jenkins

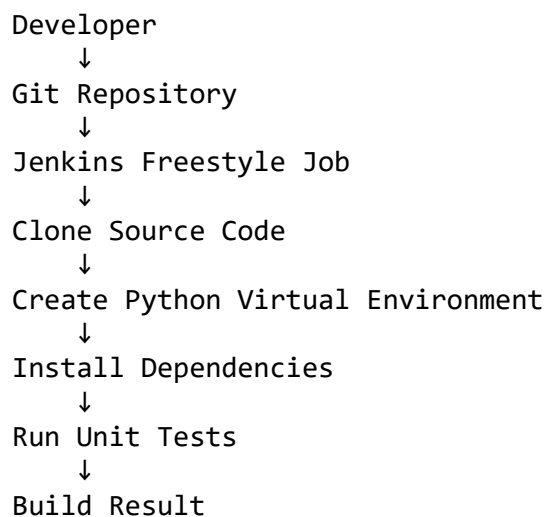
Introduction

A Jenkins Freestyle Project is one of the simplest ways to automate building, testing, and deploying applications.

In this document, we will create a Python-based Freestyle Project that:

- Retrieves source code from Git.
 - Creates a Python virtual environment.
 - Installs dependencies.
 - Executes unit tests.
 - Displays build results in Jenkins.
-

Architecture Overview



Prerequisites

Before creating a Jenkins Freestyle Project, ensure:

- Jenkins is installed and running.
- Python 3 is installed.

- Pip is installed.
 - Git is installed.
 - Access to a Git repository is available.
 - Jenkins has internet access to install Python packages.
-

Step 1: Install Required Dependencies on Jenkins Server

Connect to Jenkins Server

SSH into the Ubuntu VM where Jenkins is installed.

Example:

```
ssh ubuntu@<jenkins-server-ip>
```

Update Package Repository

Update package information:

```
sudo apt update
```

Install Required Packages

Install Python, Pip, Virtual Environment, and Git:

```
sudo apt install -y python3 python3-pip python3-venv git
```

Package Purpose

Package	Purpose
python3	Python Runtime
python3-pip	Python Package Manager
python3-venv	Virtual Environment Support
git	Source Code Management

Verify Installation

Check Python version:

```
python3 --version
```

Example:

Python 3.12.3

Check Pip version:

```
pip3 --version
```

Example:

```
pip 24.0
```

Check Git version:

```
git --version
```

Example:

```
git version 2.43.0
```

Step 2: Create a Python Project

If you already have a Python project in GitHub, GitLab, or Bitbucket, you can skip this step.

Create Project Directory

```
mkdir my_python_project  
cd my_python_project
```

Create Python Application

Create a file named:

app.py

Content:

```
def hello():  
    return "Hello, Jenkins!"  
  
if __name__ == "__main__":  
    print(hello())
```

Explanation

The function returns a simple message.

Output:

Hello, Jenkins!

Create Unit Test

Create:

test_app.py

Content:

```
import unittest
from app import hello

class TestApp(unittest.TestCase):

    def test_hello(self):
        self.assertEqual(
            hello(),
            "Hello, Jenkins!"
        )

if __name__ == "__main__":
    unittest.main()
```

Explanation

This test verifies that:

hello()

returns:

Hello, Jenkins!

Create Requirements File

Create:

requirements.txt

Content:

pytest

Purpose

Jenkins will install all dependencies listed in this file.

Project Structure

Your project should look like:

```
my_python_project/  
├── app.py  
├── test_app.py  
└── requirements.txt
```

Step 3: Push Project to Git Repository

Initialize Git:

```
git init
```

Add files:

```
git add .
```

Commit files:

```
git commit -m "Initial commit"
```

Add remote repository:

```
git remote add origin <repository-url>
```

Example:

```
git remote add origin https://github.com/user/my_python_project.git
```

Push code:

```
git push -u origin main
```

Your project is now available for Jenkins to access.

Step 4: Create Jenkins Freestyle Project

Login to Jenkins

Open browser:

`http://<jenkins-server-ip>:8080`

Create New Job

1. Click **New Item**
2. Enter Job Name

Example:

Python-Freestyle-Job

3. Select:
Freestyle Project
 4. Click **OK**
-

Step 5: Configure Source Code Management (SCM)

Scroll to:

Source Code Management

Select:

Git

Enter Repository URL

Example:

`https://github.com/user/my_python_project.git`

Configure Credentials

If the repository is private:

1. Click **Add Credentials**
 2. Add username/password or SSH key
 3. Save credentials
-

Configure Branch

Specify the branch:

main

or

master

depending on your repository.

Step 6: Configure Build Trigger (Optional)

Build triggers determine when Jenkins runs the job.

Common options:

Build Now

Manual execution.

Poll SCM

Periodically checks Git repository.

Example:

H/5 * * * *

Checks every 5 minutes.

GitHub Webhook

Automatically triggers builds whenever code is pushed.

Recommended for CI/CD environments.

Step 7: Configure Build Steps

Scroll to:

Build

Click:

Add Build Step

Select:

Execute Shell

Build Script

Paste:

```
#!/bin/bash
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install --upgrade pip
```

```
pip install -r requirements.txt
```

```
python3 -m unittest discover
```

Script Explanation

Create Virtual Environment

```
python3 -m venv venv
```

Creates an isolated Python environment.

Activate Virtual Environment

`source venv/bin/activate`

Activates the environment.

Upgrade Pip

`pip install --upgrade pip`

Installs the latest Pip version.

Install Dependencies

`pip install -r requirements.txt`

Installs packages listed in:

`requirements.txt`

Run Unit Tests

`python3 -m unittest discover`

Automatically discovers and executes all test files.

Step 8: Save Configuration

Click:

Save

Jenkins stores the project configuration.

Step 9: Build the Project

Click:

Build Now

Jenkins will start the build process.

Step 10: Verify Build Execution

After build completion:

1. Click Build Number
2. Click Console Output

Example:

Started by user admin

Cloning repository...

Creating virtual environment...

Installing dependencies...

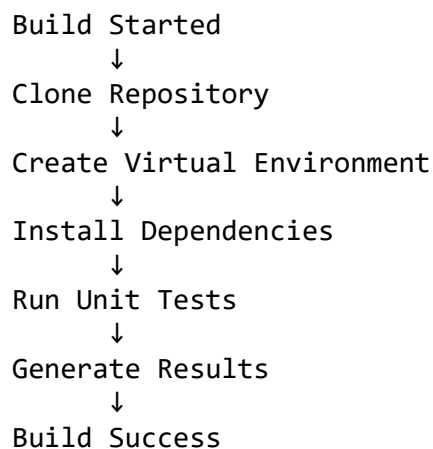
Running tests...

Ran 1 test in 0.001s

OK

Finished: SUCCESS

Expected Build Workflow



Step 11: Common Troubleshooting

Issue: Virtual Environment Activation Fails

Error:

source: not found

Solution:

```
. venv/bin/activate
```

Use dot notation instead of source.

Issue: Requirements File Missing

Error:

Could not open requirements file

Solution:

Verify:

requirements.txt

exists in the repository root.

Issue: Git Clone Failure

Error:

Authentication failed

Solution:

- Verify repository URL.
 - Verify Jenkins credentials.
 - Verify Git access permissions.
-

Issue: Python Not Found

Error:

python3: command not found

Solution:

Install Python:

```
sudo apt install python3
```

Issue: Test Failure

Error:

FAILED (failures=1)

Solution:

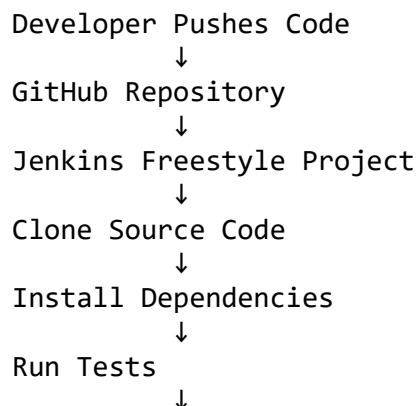
- Review test code.
 - Check application logic.
 - Verify expected results.
-

Build Status Indicators

Status	Meaning
SUCCESS	Build completed successfully
FAILURE	Build failed
UNSTABLE	Tests failed but build completed
ABORTED	Build manually stopped

Real-World Freestyle Workflow

In a real DevOps environment:



Generate Reports



Notify Team

Conclusion

A Jenkins Freestyle Project provides a simple way to automate Python application builds and testing.

The process includes:

1. Installing Python and Git.
2. Creating a Python project.
3. Storing code in Git.
4. Creating a Jenkins Freestyle Job.
5. Configuring Source Code Management.
6. Creating a virtual environment.
7. Installing dependencies.
8. Running automated tests.
9. Reviewing build results.

This serves as a foundation for implementing Continuous Integration (CI) pipelines using Jenkins.